



universität
wien

Internationalise your studies!

ERASMUS+ Studienaufenthalte

an einer europäischen Partneruniversität

ERASMUS+ Praktika

Selbst organisiertes studienbezogenes
Praktikum im europäischen Ausland



Non-EU Student ExchangeProgram

Studienaufenthalt an einer Partneruniversität in u. a. Australien,
Kanada, USA, Afrika, Asien, Lateinamerika, Israel

Kurzfristige wissenschaftliche Auslandsstipendien (KWA)

Förderung für Forschungsarbeiten im Ausland im
Rahmen der Abschlussarbeit - weltweit

Circle U.

Vielfältige Mobilitätsangebote zwischen den
Circle U.-Partneruniversitäten



Hole dir weitere
Infos im International Office
oder nimm an unseren
Infoveranstaltungen teil!

international.univie.ac.at



Parallel computing in Python

Kristina Kislyakova

University of Vienna, Department of Astrophysics, Vienna, Austria

Time plan and assignments

This lecture briefly explains how the **parallel computing** can be done using Python and contains **some examples**.

Literature:

Jan Palach, **Parallel Processing with Python** (PACKT Publishing, 2014)

+ various internet sources (e.g.,

[https : //wiki.python.org/moin/ParallelProcessing](https://wiki.python.org/moin/ParallelProcessing), Stack Overflow, etc.)

Time plan and assignments

This lecture briefly explains how the **parallel computing** can be done using Python and contains **some examples**.

Literature:

Jan Palach, **Parallel Processing with Python** (PACKT Publishing, 2014)

+ various internet sources (e.g.,

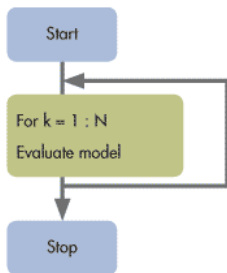
[https : //wiki.python.org/moin/ParallelProcessing](https://wiki.python.org/moin/ParallelProcessing), Stack Overflow, etc.)

If really necessary, use the Force **the power of the collective mind** at the Internet (e.g., **Stack Overflow**)

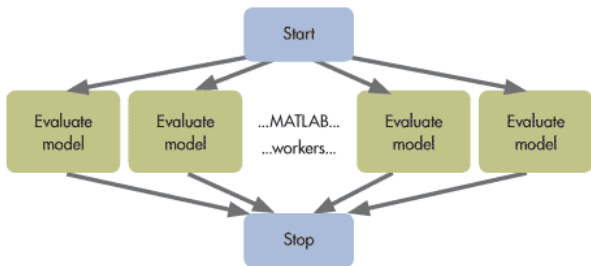
However, **google** the solution before creating a new thread, because the majority of the questions have already been answered and can be found online.

Parallel computing

Serial Approach



Parallel Approach



Questions to ask before parallelizing

- Can my task be **easily scaled**?
- Are there **interdependencies** in my data that would make parallelization inefficient?
- Have I exhausted **other means** to accelerate my code?

→ parallelization is not always necessary or efficient, but for many scientific tasks it is absolutely unavoidable (for instance, for DSMC codes with their millions of particles)

Efficient coding in Python: tips & tricks

Avoid loops! For instance:

```
a = np.linspace(1, 1000, 10000000)
b = np.zeros_like(a)
for i in range(a.shape[0]):
    b[i] = np.sqrt(a[i])
```

Execution time 0.00958299636841 s

Efficient coding in Python: tips & tricks

Avoid loops! For instance:

```
a = np.linspace(1, 1000, 10000000)
b = np.zeros_like(a)
for i in range(a.shape[0]):
    b[i] = np.sqrt(a[i])
```

Execution time 0.00958299636841 s

```
a = np.linspace(1, 1000, 10000000)
b = np.sqrt(a)
```

Execution time 9.29832458496e-05 s → not only easier to read, **but also faster**. Python is slow to begin with, but extensive usage of loops makes it even slower

Efficient coding in Python: tips & tricks

Use built-in functions and libraries when possible:

Slow

```
total = 0
for x in range(1000):
    total += x
```

Fast

```
total = sum(range(1000))
```

Efficient coding in Python: tips & tricks

Minimize function calls inside loops:

```
import math
```

```
# Slow
```

```
result = [math.sqrt(x) for x in range(1000)]
```

```
# Faster
```

```
sqrt = math.sqrt
```

```
result = [sqrt(x) for x in range(1000)]
```

Efficient coding in Python: tips & tricks

Use list comprehensions:

Slow

```
result = []  
for x in range(1000):  
    result.append(x**2)
```

Fast

```
result = [x**2 for x in range(1000)]
```

Efficient coding in Python: tips & tricks

Use generators for large data sets:

Generator

```
def squares_gen(n):  
    for i in range(n):  
        yield i**2
```

List comprehension (uses more memory)

```
squares = [i**2 for i in range(1000000)]
```

Efficient coding in Python: tips & tricks

Use appropriate data structures:

```
items = [1, 2, 3]
if 2 in items:
    pass
```

```
# Fast
items = {1, 2, 3}
if 2 in items:
    pass
```

Efficient coding in Python: tips & tricks

Avoid repeated calculations:

Slow

```
result = [(x**2 + x**2) for x in range(1000)]
```

Fast

```
result = [2 * (x**2) for x in range(1000)]
```

Efficient coding in Python: tips & tricks

Profile and find out where the worst bottlenecks are:

```
python -m cProfile my_script.py
```

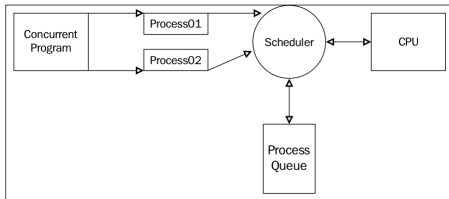
Efficient coding in Python: tips & tricks

Example of a horrible code and the same code slightly optimized (with timings).

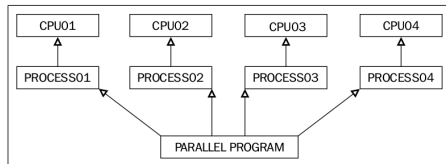
Forms of parallelization

- **Concurrent programming**: a program dispatches several workers and these workers **dispute to use the CPU** to run a task
- **Parallel programming**: workers run specific tasks simultaneously in a **multicore environment** without the need for concurrency amongst them to access a CPU.
- **Distributed programming**: sharing the processing by exchanging data through messages between machines (nodes) that are **physically separated**.

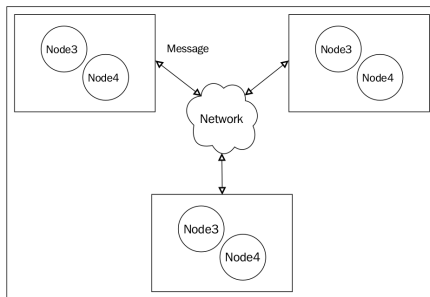
Forms of parallelization



Concurrent programming scheme.



Parallel programming scheme.



Distributed programming scheme.

Communication in parallel programming

Data exchange between workers is necessary. Most common types:

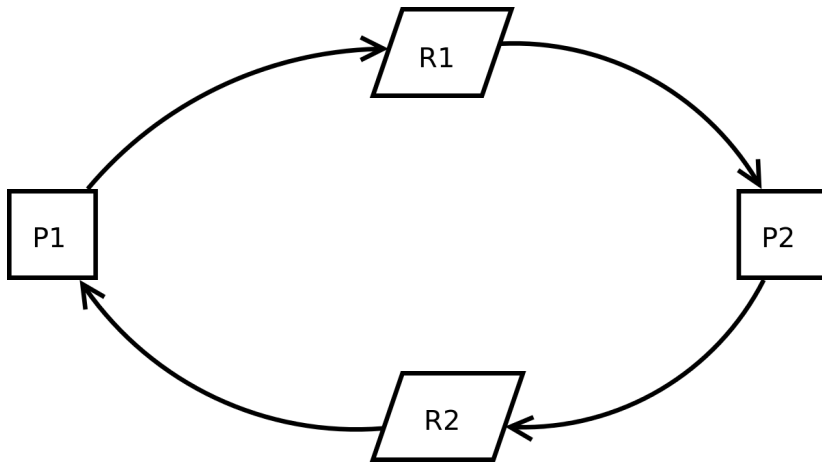
- **Shared state:** workers use the same resources (can lead to problems if one worker overwrites the results of another)
- **Message passing:** each worker has a separate piece of data (**advantage:** no concurrence for data and related problems, **disadvantage:** more memory and resources necessary)

Common problems in parallel programming

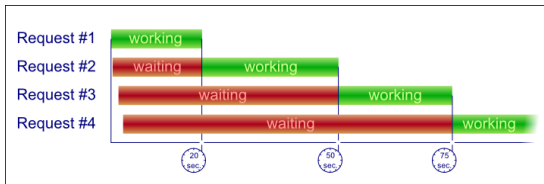
- **Deadlock:** two or more workers indefinitely wait for each other
- **Race conditions:** wrongly synchronized access to data can lead to erroneous results
- **Starvation:** unfair ranking of one or more processes → some processes or workers have no resources

Deadlock

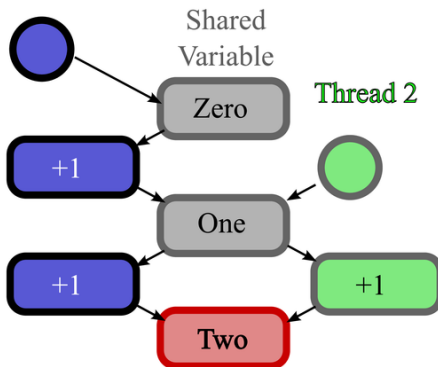
Processes/threads waiting for each other indefinitely



Race conditions



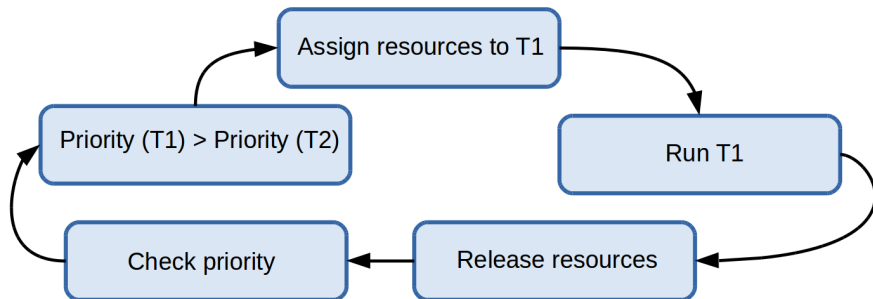
Thread 1



Starvation

T1: a thread with high priority

T2: a thread with low priority



T2 can eventually time out (or the system gets into a deadlock, if at any point T1 requires the results from T2)

Processes VS threads

Both processes and threads are independent sequences of execution. The typical difference is that threads (of the same process) run in a **shared memory space**, while processes run in **separate memory spaces**.

Processes VS threads

Both processes and threads are independent sequences of execution. The typical difference is that threads (of the same process) run in a **shared memory space**, while processes run in **separate memory spaces**.

Python allows to work both with **processes** and **threads**:

- The *threading* module for threads
- The *multiprocessing* module for processes
- The *Celery* module for tasks performed at different machines connected by a network

However: be cautious with threads in Python, is difficult to process them using GIL (Global Interpreter Lock)

Designing a parallel code

The algorithm:

- 1 Decompose the task into parallelizable subtasks (divide and conquer!)
- 2 Decompose data
- 3 Pipeline: break large tasks into smaller independent tasks and run them in a parallel manner
- 4 Process and map the data

THREADING MODULE

More about threads

Threads are different execution lines in a process. They belong to the **same process** and share the **same memory space**.

Advantages:

- Fast speed of communication
- Creation of a thread less costly
- Data locality (cache memory)

Disadvantages:

- Shared memory usage can lead to errors which are difficult to solve
- Limited flexibility and scalability of algorithms

Kernel threads: created by the OS. **User threads:** created by the user.

More about threads

Lifecycle and possible states of a thread:

- 1 Creation
- 2 Execution
- 3 Ready
- 4 Blocked
- 5 Concluded

Python offers two modules to handle threads: `_thread` module and `threading` module. The former is more low level, the latter has more friendly interface than the former.

Daemon threads

“Normal” threads:

- Programs implicitly wait to exit until all threads have completed their work

Daemon threads:

- The thread runs without blocking the main program from exiting

Using **daemon threads** is useful for services where there may not be an easy way to interrupt the thread or where letting the thread die in the middle of its work does not lose or corrupt data.

GIL and threads in Python

Python uses **reference counting** for memory management. It means that objects created in Python have a **reference count variable** that keeps track of the number of references that point to the object.

```
>>> import sys
>>> a = []
>>> b = a
>>> sys.getrefcount(a)
3
```

GIL protects the reference counting from race conditions and only allows **one thread at a time** to hold control of the Python interpreter.

The usage of GIL leads to **only I/O-bound threads** having a decrease in execution time if parallelized (opposite to **CPU-bound threads**)

Examples & more about threads

<https://pymotw.com/3/threading/index.html>

<https://realpython.com/intro-to-python-threading/>

MULTIPROCESSING MODULE

More about processes

Processes in OS are containers for programs in execution and their resources.

Advantages and **disadvantages** are the opposite to **threads**. **Processes** don't share the **same memory space**.

Advantages:

- No shared memory usage → **fewer** nasty errors in comparison to threads
- Flexibility and scalability of algorithms can be achieved much easier

Disadvantages:

- Slow speed of communication
- Creation of a process is quite costly
- Data non-locality (no cache memory)

More about processes

Associated information and resources:

- ① Process ID
- ② Program counter
- ③ I/O information
- ④ Memory allocation
- ⑤ CPU scheduling
- ⑥ Priority
- ⑦ Current state
- ⑧ CPU registry

More about processes

Associated information and resources:

- 1 Process ID
- 2 Program counter
- 3 I/O information
- 4 Memory allocation
- 5 CPU scheduling
- 6 Priority
- 7 Current state
- 8 CPU registry

State of a process:

- 1 **Running**: The process is making use of the CPU
- 2 **Ready**: The process that was waiting in the processes queue is now ready to use the CPU
- 3 **Waiting**: The process is waiting for some I/O operation related to the task it was executing

More about processes

Associated information and resources:

- 1 Process ID
- 2 Program counter
- 3 I/O information
- 4 Memory allocation
- 5 CPU scheduling
- 6 Priority
- 7 Current state
- 8 CPU registry

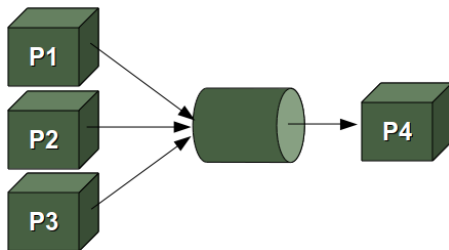
State of a process:

- 1 **Running**: The process is making use of the CPU
- 2 **Ready**: The process that was waiting in the processes queue is now ready to use the CPU
- 3 **Waiting**: The process is waiting for some I/O operation related to the task it was executing

In Python, processes are handled by the *multiprocessing* module

Understanding *multiprocessing Pipe*

A **pipe** consists of a mechanism that establishes communication between two endpoints (two processes in communication), i.e., it creates a **communication channel**. Both the **Pipe** and **Queue** mechanisms make use of the **message passing** paradigm, which spares users from the need to use synchronization mechanisms.



Understanding *multiprocessing Queue*

Queue is another method in Python used to **establish communication**.

Queue VS Pipe:

- A Pipe() can only have two endpoints
- A Queue() can have multiple producers and consumers
- If you need more than two points to communicate, use a Queue()
- If you need absolute performance, a Pipe() is much faster because Queue() is built on top of Pipe()

Pipes are faster, but **Queues** are process- and thread- safe. Pipes are not: if two different processes try to read from or write to the same end of a pipe, bad things happen.

Queue: simple example

The Queue class implements a basic first-in, first-out container. Elements are added to one “end” of the sequence using `put()`, and removed from the other end using `get()`.

```
import queue

q = queue.Queue()

for i in range(5):
    q.put(i)

while not q.empty():
    print (q.get())
```


Queue: simple example

Output:

```
$ python Queue_fifo.py
```

```
0
1
2
3
4
```

For last-in, first-out ordering (normally associated with a stack data structure), use $q = \text{Queue.LifoQueue}()$

Simple example of multiprocessing

```
import multiprocessing

def worker(num):
    """thread worker function"""
    print ('Worker:', num)
    return

if __name__ == '__main__':
    jobs = []
    for i in range(5):
        p = multiprocessing.Process(target=worker,
                                    args=(i,))

        jobs.append(p)
        p.start()
```

How to get the program's execution time

Knowing the execution time is **necessary** to determine which parts of the code are the slowest (i.e., should be rewritten or parallelized) and if parallelization speeds up the program at all (if not implemented correctly, it actually does **exactly the opposite!**)

A very easy way of measuring the execution time in Python: the **time module** (also see `cProfile`, `line_profiler`, and `timeit`).

Calculating execution time: example

```
import numpy as np
import time

def square(x):
    return x*x # some function

if __name__ == '__main__':

    dataset = np.linspace(0,100.,10) # create data
    start_time = time.time() # get time before
    result = square(dataset) # do something

    # get the time of the execution in seconds
    print("Execution_time_{}_seconds"
          % (time.time() - start_time))
```

Parallelizing a function using Pool

```
from multiprocessing import Pool

def square(x):
    # calculate the square of the value of x
    return x*x

if __name__ == '__main__':

    # Define the dataset
    dataset = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    # Output the dataset
    print ('Dataset:␣' + str(dataset))
```

```
# Run this with a pool of 5 agents having a
# chunksize of 3 until finished
agents = 5
chunksize = 3
with Pool(processes=agents) as pool:
    result = pool.map(square, dataset, chunksize)

# Output the result
print ('Result:␣␣' + str(result))
```

Running this code should yield the following output:

```
$ python3 pool_multiprocessing.py
Dataset: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Result:  [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

More examples

Go to the **Python's module of the week**:

<https://pymotw.com/3/multiprocessing/basics.html>

Communication between processes:

<https://pymotw.com/3/multiprocessing/communication.html>

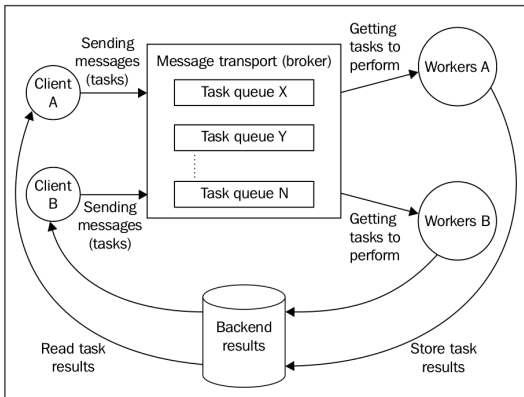
Also see

<https://docs.python.org/3/library/multiprocessing.html>

CELERY MODULE

Celery

The **Celery** framework works with the concept of distribution of work units (tasks) by exchanging messages among the machines **that are interconnected as a network**, or local workers.



The Celery architecture

Conclusions

- Parallelization is sometimes unavoidable
- One should implement it correctly, otherwise the code will become **slower** instead of faster
- Python-specific: **beware of GIL!**
- If you have a large scale task: think about using **Fortran or C(++)** instead of Python
- But do not discard Python completely: it is absolutely great for model testing and toy model creation